

Bayesian Networks

Structured Representation and Reasoning under Uncertainty

From Probability to Bayesian Networks I

We can:

- represent uncertainty using probability distributions;
- compute conditional probabilities;
- update beliefs using Bayes' rule.

A natural question now arises:

The problem

How can we represent a probability distribution over many variables without explicitly storing the entire joint distribution?

Bayesian networks provide one answer.

Why the Full Joint Distribution is a Problem I

Suppose we have n binary random variables.

The full joint distribution contains

$$2^n$$

possible assignments.

Number of variables	Joint assignments
5	32
10	1,024
20	1,048,576
30	1,073,741,824

Why the Full Joint Distribution is a Problem II

Most real-world variables are **not independent of everything else**.
But neither are they dependent on everything else.

Key idea

Exploit the structure of the dependencies.

A Structured View of the Joint Distribution I

Suppose we have variables

$$X_1, X_2, \dots, X_n.$$

The chain rule always allows us to write

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid x_1, \dots, x_{i-1}).$$

This is exact.

But the conditional probabilities may still involve **many variables**.

Question

Can we replace these large conditioning sets by much smaller ones?

Bayesian networks do exactly this by exploiting **conditional independence**.

What is a Bayesian Network? I

A Bayesian network consists of:

- ① a **directed acyclic graph (DAG)**;
- ② a probability distribution associated with each node, conditioned on its parents.

The graph contains:

- one node for each random variable;
- directed edges representing the dependency structure;
- no directed cycles.

For each variable X we specify

$$P(X \mid \text{Parents}(X)).$$

A Bayesian Network is a Graph I

For example:



The graph says that B and E are parents of A :

$$B \rightarrow A \leftarrow E.$$

Therefore the local probability distributions are

$$P(B), \quad P(E), \quad P(A \mid B, E).$$

An Important Qualification About Arrows I

It is tempting to interpret

$$X \longrightarrow Y$$

as saying:

“ X causes Y .”

This interpretation is **not automatically justified**.

In a general Bayesian network, an edge specifies a probabilistic dependency structure.
Thus

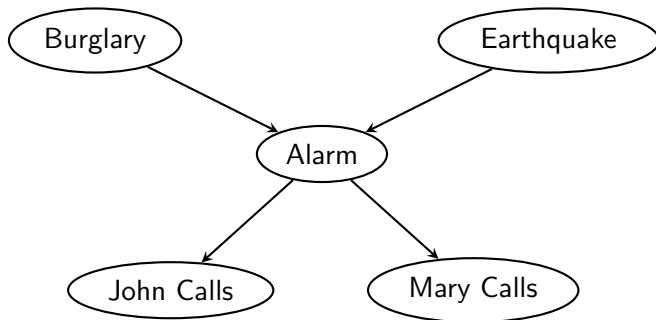
$$X \longrightarrow Y$$

means that the model allows the distribution of Y to depend on X .

When additional causal assumptions are made, the arrows can also be given a causal interpretation.

The Alarm Bayesian Network I

Consider a simple security system.



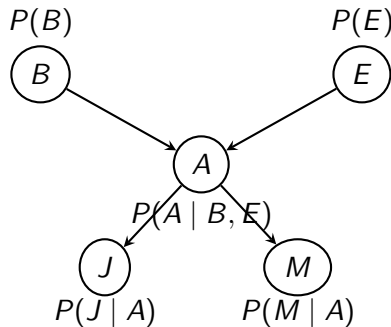
The Alarm Bayesian Network II

We use the abbreviations

B = Burglary, E = Earthquake, A = Alarm,

J = John Calls, M = Mary Calls.

The Alarm Network and Its CPTs I



The Alarm Network and Its CPTs II

The graph specifies the **conditioning structure**.

For example,

$$B \rightarrow A \leftarrow E \quad \Longrightarrow \quad P(A \mid B, E).$$

The CPTs specify the corresponding numerical probabilities.

The Key Factorisation I

For a Bayesian network with variables $X_1, \dots, X_n$,

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{Parents}(X_i))$$

Each factor is **local**:

- it involves one variable;
- it is conditioned only on its parents.

The payoff

A potentially enormous joint distribution is represented using a collection of much smaller local distributions.

Why Does the Factorisation Work? I

Recall the chain rule:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid x_1, \dots, x_{i-1}).$$

A Bayesian network replaces

$$P(x_i \mid x_1, \dots, x_{i-1})$$

by

$$P(x_i \mid \text{Parents}(X_i)).$$

This is justified by the conditional-independence assumptions encoded by the network.

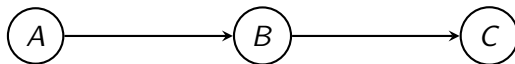
Why Does the Factorisation Work? II

Local Markov condition

Each variable is conditionally independent of its non-descendants given its parents.

Conditional Independence: A Chain I

Consider



Without observing B , A and C may be dependent.

Once B is known,

$$A \perp C \mid B$$

or equivalently,

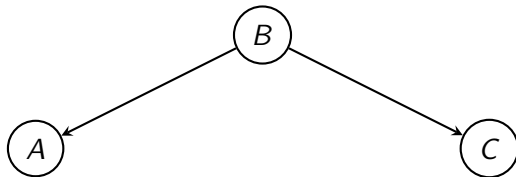
$$P(C \mid A, B) = P(C \mid B).$$

Intuition

Knowing the intermediate variable blocks the information flow from A to C .

Conditional Independence: A Fork I

Consider



B is a common influencer of A and C .

Typically,

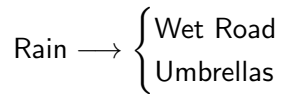
$$A \not\perp C.$$

But after observing B ,

$$\boxed{A \perp C \mid B}.$$

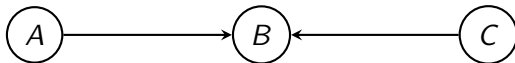
Conditional Independence: A Fork II

Example:



Conditional Independence: A Collider I

Consider



Here B is a **collider**.

Before observing B ,

$$A \perp C.$$

But conditioning on B can create dependence:

$$A \not\perp C \mid B.$$

Example:

Burglary $\longrightarrow$ Alarm $\longleftarrow$ Earthquake.

A Useful Mental Model I

The three structures provide an important intuition:

Structure	Effect of conditioning
Chain	blocks information
Fork	blocks information
Collider	can open information

These local patterns generalise to larger networks.

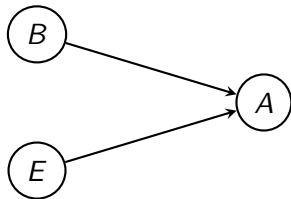
Determining whether two variables are conditionally independent given observed variables is the basis of

d-separation.

We will use the intuition here without requiring the full d-separation algorithm.

The Graph Determines the CPT I

Consider the Alarm node.



B	E	$P(A = T \mid B, E)$
t	t	0.95
t	f	0.94
f	t	0.29
f	f	0.001

The graph determines that we need

$$P(A \mid B, E).$$

The CPT supplies the numerical values.

Constructing a Bayesian Network I

A practical construction procedure is:

- ① Identify the relevant random variables.
- ② Choose an ordering for the variables.
- ③ Identify the relevant dependencies.
- ④ Construct a DAG.
- ⑤ Specify the local conditional distributions.

For each variable X we specify

$$P(X \mid \text{Parents}(X)).$$

The resulting collection of local distributions defines a joint probability distribution.

Choosing the Network Structure I

Some useful principles are:

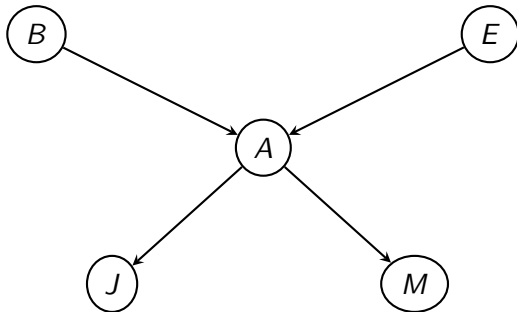
- Prefer a **sparse** network.
- Represent important dependencies explicitly.
- Avoid unnecessary edges.
- Use domain knowledge where available.
- Choose variables that are useful for the intended inference tasks.

Engineering perspective

There may be more than one reasonable network representation of a domain.

The Full Joint Distribution from the Alarm Network I

The Alarm network is



Therefore,

$$P(B, E, A, J, M) = P(B)P(E)P(A | B, E)P(J | A)P(M | A)$$

The graph gives us the factorisation of the joint distribution.

How Much Representation Do We Save? I

Suppose all five variables are binary.
The full joint distribution contains

$$2^5 = 32$$

possible assignments.

The Bayesian network requires:

Distribution	Independent probabilities
$P(B)$	1
$P(E)$	1
$P(A \mid B, E)$	4
$P(J \mid A)$	2
$P(M \mid A)$	2
Total	10

How Much Representation Do We Save? II

Important

The savings come from exploiting conditional independence, not from approximating the distribution.

But Bayesian Networks Are Not Automatically Small I

Suppose X is binary and has k binary parents.
The CPT has

$$2^k$$

conditioning cases.

Therefore the representation is exponential in the number of parents of a node.

A BN is particularly useful when the graph is relatively sparse:

many variables but **few parents per variable.**

This is one reason why network structure matters.

A Bayesian Network Supports Many Queries I

Once the network is specified, we can ask many different probabilistic questions.
For example:

$$P(B)$$

$$P(B \mid J)$$

$$P(B \mid J, M)$$

$$P(E \mid A)$$

$$P(A \mid B)$$

A Bayesian Network Supports Many Queries II

The same network can therefore support both **prediction** and **diagnosis**.

Key point

The network is a reusable model of the domain, not a model for only one particular question.

The Basic Inference Problem I

Suppose some variables have been observed.

Call these observations the **evidence** e .

We want the posterior distribution over a query variable X :

$$P(X \mid e).$$

Variables that are neither query variables nor evidence variables are **hidden variables**.

For example,

$$P(B \mid J, M)$$

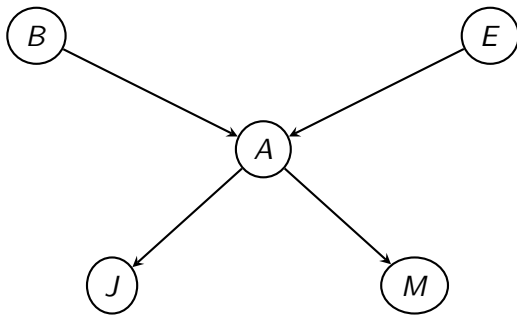
asks:

What is the probability of burglary given that John and Mary called?

Inference in a Bayesian Network I

Suppose John and Mary have both called.
We want

$$P(B \mid J, M).$$



Inference in a Bayesian Network II

- Query variable: B
- Evidence variables: J, M
- Hidden variables: E, A

Inference and the Sum Rule I

From probability theory,

$$P(X \mid e) = \frac{P(X, e)}{P(e)}.$$

If $\mathbf{Y}$ denotes the hidden variables,

$$P(X, e) = \sum_{\mathbf{Y}} P(X, e, \mathbf{Y}).$$

Therefore,

$$P(X \mid e) = \alpha \sum_{\mathbf{Y}} P(X, e, \mathbf{Y})$$

where α is a normalising constant.

Inference and the Sum Rule II

Inference therefore requires:

- ① summing out variables we do not care about;
- ② normalising the resulting probabilities.

Inference by Enumeration I

The simplest approach is:

- 1 enumerate all assignments to the hidden variables;
- 2 calculate the corresponding joint probabilities;
- 3 add the appropriate terms;
- 4 normalise.

For example,

$$P(B \mid J, M) = \alpha \sum_E \sum_A P(B, E, A, J, M).$$

Using the BN factorisation,

$$P(B, E, A, J, M) = P(B)P(E)P(A \mid B, E)P(J \mid A)P(M \mid A).$$

Worked Inference: The Alarm Network I

For $B = T$,

$$P(b, j, m) = \sum_E \sum_A P(b)P(E)P(A \mid b, E)P(j \mid A)P(m \mid A).$$

Similarly,

$$P(\neg b, j, m) = \sum_E \sum_A P(\neg b)P(E)P(A \mid \neg b, E)P(j \mid A)P(m \mid A).$$

Using the standard Alarm-network probabilities gives

$$P(b, j, m) \approx 0.000592$$

and

$$P(\neg b, j, m) \approx 0.001494.$$

Worked Inference: The Alarm Network II

After normalisation,

$$P(B \mid J, M) \approx 0.284$$

and

$$P(\neg B \mid J, M) \approx 0.716.$$

What Enumeration Teaches Us I

Enumeration is conceptually simple.

But if there are h binary hidden variables, we may need to consider

$$2^h$$

assignments.

There is another problem:

Repeated computation

Many intermediate expressions are evaluated repeatedly.

For example, the same factor involving a variable may occur in several branches of the enumeration.

What Enumeration Teaches Us II

Question

Can we organise the computation so that repeated subcomputations are performed only once?

Variable Elimination: The Basic Idea I

Instead of enumerating complete assignments, we can manipulate the local factors directly.
The basic operation is:

multiply relevant factors, then sum out a variable.

For example,

$$\sum_A f(A)g(A)h(A)$$

can be evaluated once and reused rather than recomputed independently in many branches.

Key idea

The graph allows inference algorithms to exploit the same structure that made the representation compact.

The Markov Blanket I

For a variable X , its **Markov blanket** consists of:

- ① the parents of X ;
- ② the children of X ;
- ③ the other parents of its children.

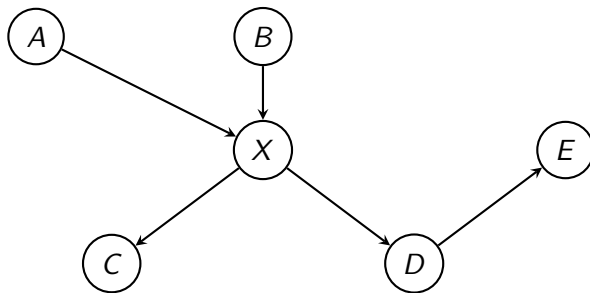
The important property is

X is conditionally independent of the rest of the network given its Markov blanket.

The Markov blanket therefore identifies the variables that provide the information locally relevant to X .

The Markov Blanket: Intuition I

Consider



The Markov Blanket: Intuition II

The Markov blanket of X contains

$$\{A, B, C, D\}.$$

Once these variables are known, variables outside the blanket provide no additional information about X .

Naive Bayes: A Simple Bayesian Network I

Consider a classification problem.

Let

$$Y = \text{class}$$

and let

$$X_1, \dots, X_d$$

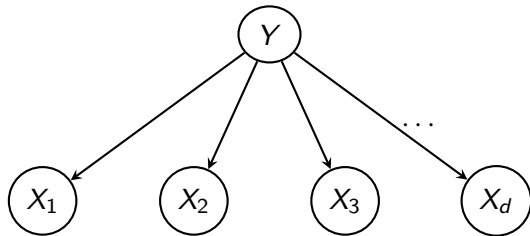
be observed features.

Naive Bayes assumes

$$X_i \perp X_j \mid Y \quad (i \neq j).$$

The corresponding network is

Naive Bayes: A Simple Bayesian Network II



The features are conditionally independent given the class.

Naive Bayes and Bayes' Rule I

Using Bayes' rule,

$$P(Y \mid x_1, \dots, x_d) \propto P(x_1, \dots, x_d \mid Y)P(Y).$$

The Naive Bayes assumption gives

$$P(x_1, \dots, x_d \mid Y) = \prod_{i=1}^d P(x_i \mid Y).$$

Therefore

$$P(Y \mid x_1, \dots, x_d) \propto P(Y) \prod_{i=1}^d P(x_i \mid Y)$$

Naive Bayes and Bayes' Rule II

Naive Bayes is therefore an example of a Bayesian network whose structure makes inference particularly simple.

What a Bayesian Network Gives Us I

A Bayesian network combines three ideas:

① **Probability**

Represent uncertainty quantitatively.

② **Structure**

Represent dependencies and conditional independences.

③ **Inference**

Use the structure to compute posterior beliefs.

Thus:

Probability + Structure $\longrightarrow$ Structured Reasoning
--

Limitations I

Bayesian networks are powerful, but they do not make probabilistic inference automatically easy.

- CPTs can become large.
- Inference can be computationally expensive.
- The quality of the model depends on its assumptions.
- Network structure may be difficult to specify.
- Probabilities may be difficult to obtain.

Nevertheless, exploiting structure can make problems that would otherwise be infeasible computationally manageable.

Summary I

A Bayesian network consists of

DAG + local probability distributions

The DAG represents dependency and conditional-independence structure.
The network defines the joint distribution

$$P(x_1, \dots, x_n) = \prod_i P(x_i \mid \text{Parents}(X_i))$$

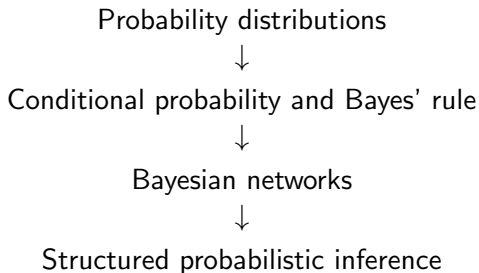
Inference asks questions such as

$$P(X \mid e).$$

The graph allows us to reason about such queries without explicitly constructing the full joint distribution.

From Probability to Probabilistic AI I

We have now moved through three levels:



The next question is:

Next step

How can we learn the structure and probabilities of a Bayesian network from data?

From Probability to Probabilistic AI II

This takes us from

probabilistic representation

to

probabilistic learning.